

Smart Ground – IoT System für Schiessplatz-Gerätemanagement

IoT-basierte Digitalisierung und Verwaltung von Schiessplatz-Geräten (Wurfmaschinen, LEDs, Sensoren) via MQTT, Spring Boot, Vue.js & REST-API

Version 1.0-dev | Mai 2026

1. Projektübersicht

Smart Ground ist ein IoT-Verwaltungssystem für Schiessplätze, das Wurfmaschinen, LED-Anlagen und Sensoren über eine zentrale REST-API und MQTT-Broker verwaltet. Das System besteht aus drei unabhängigen Komponenten:

- **Backend (Spring Boot 4)**: REST-API, MQTT-Integration, Datenhaltung
- **Frontend (Vue.js 3)**: Device-Management und Wettkampf-Verwaltung
- **SmartBox Firmware (MicroPython)**: Raspberry Pi Pico 2W für lokale Gerätesteuerung

Kernfunktionen:

- **Zentrale Geräte-Verwaltung**: CRUD für Geräte, Gerätetypen, Firmware-Konfigurationen
- **MQTT-basierte Kommunikation**: Discovery, Config-Push, Command-Handling, Status-Updates
- **Wettkampf-Verwaltung**: League, Bracket, Knockout Competition Formats
- **Real-Time Updates**: Server-Sent Events (SSE) für Live-Status von SmartBoxen und Geräten
- **Authentifizierung**: JWT-basiert mit Rollen (ADMIN, SHOOTER)

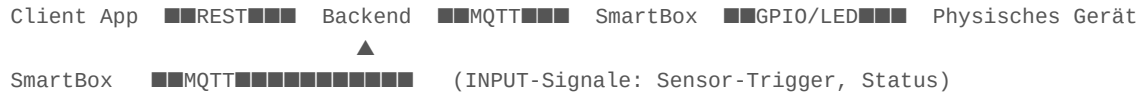
2. Technischer Kontext

Komponente	Technologie
SmartBox-Hardware	Raspberry Pi Pico 2W
SmartBox-Firmware	MicroPython 1.23+ + MQTT (umqtt.simple)
MQTT-Broker	Eclipse Mosquitto 2 (Docker)
Backend	Spring Boot 4, Java 25
Datenbank	PostgreSQL (prod), H2 (tests)
Schema-Verwaltung	Liquibase (aktiviert ab v1.0)
Backend-API	REST (JSON), OpenAPI-generiert
Web-Frontend	Vue 3 Composition API, Vite 8, Pinia
Realtime	Server-Sent Events (SSE) auf <code>/api/events`</code>
Containerisierung	Docker & Docker Compose

3. Systemarchitektur

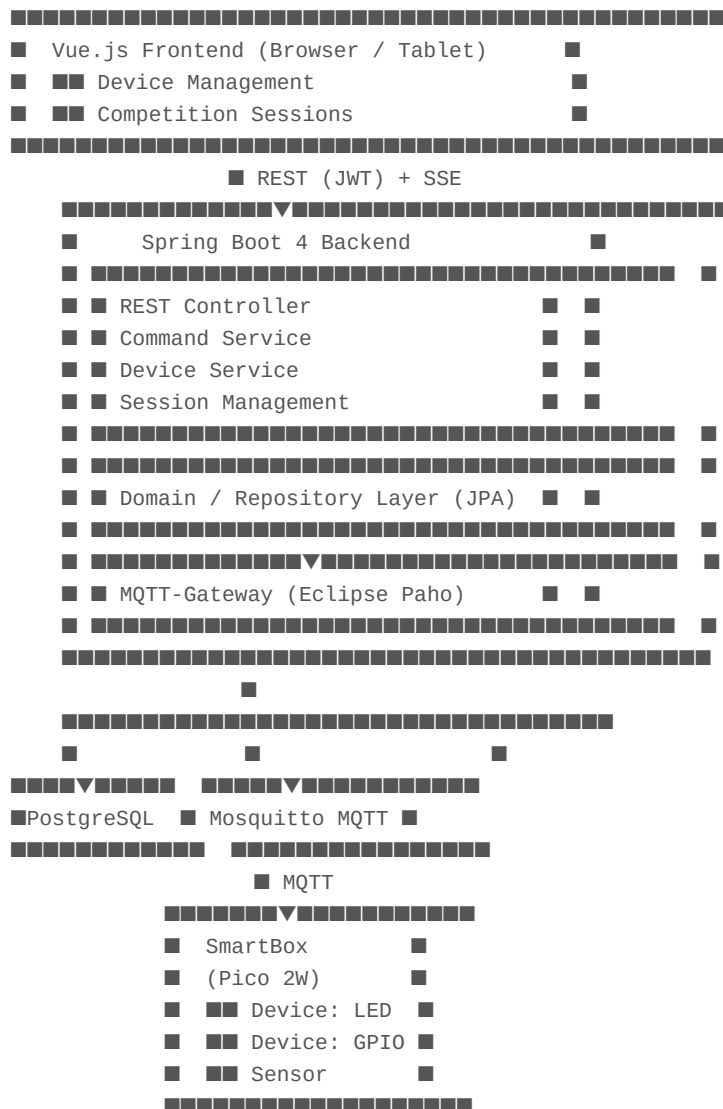
Überblick

Das Backend ist der zentrale Koordinator aller Kommunikation:



****Architektur-Prinzip****: Das Backend ist alleinige Autorität für Sicherheit, Datenhaltung und MQTT-Vermittlung. SmartBoxen sind identifiziert über ****MAC-Adresse**** (als MQTT Client ID und Topic-Segment).

Komponentendiagramm



Docker Compose Services

	Port	Beschreibung
	8080	Spring Boot 4 REST-API
	5432	PostgreSQL 18
	1883/9001	Eclipse Mosquitto 2 (MQTT / WebSocket)
	80/443	Vue.js SPA via Nginx

4. Datenmodell

Kern-Entitäten

Entität	Funktion
SmartBox	Physisches Steuergerät (Pico 2W), identifiziert durch UUID (DB) und MAC-Adresse (MQTT).
FirmwareConfig	Fähigkeits-Registry für `(version, box_type)` Paar.
SignalType	Ein einzelnes Kommando, das eine Firmware-Version senden oder empfangen kann. F
DeviceTypeGroup	Logische Gruppierung (z.B. "Wurfmaschine", "LED").
DeviceType	Spezifischer Gerätetyp, 1:1 zu `SignalType`. Trägt `signalDurationMs` (default 100 ms)
Device	Physisches Gerät registriert auf einer SmartBox. Hat `pinConfig` (Pin-Mapping JSON) u
Range	Schiessplatz / Schiessbahn. Geräte können einem Range zugewiesen werden.

Wettkampf-Entitäten

Entität	Funktion
Session	Ein Wettkampf-Ereignis (Status: SETUP, ACTIVE, COMPLETED, CANCELLED). Form
Group	Sammlung von Spielern / Schützen (z.B. "Round 1 Group A").
SessionPlayer	Ein Schütze in der Session (Typ: USER oder GUEST).
PlayerResult	Punkte und Genauigkeitsdaten für einen Spieler in einer Session.
Leaderboard	Live-Ranking nach Punkten, Genauigkeit, Win-Ratio.

5. MQTT-Konvention

Alle Topics verwenden die SmartBox ****MAC-Adresse**** als Topic-Segment:

```

smartboxes/discovery      # SmartBox → Backend: Registrierungs-Payload
smartboxes/{mac}/status   # SmartBox → Backend: Heartbeat / Status
smartboxes/{mac}/config   # Backend → SmartBox: Geräte- und GPIO-Konfiguration
smartboxes/{mac}/config/ack # SmartBox → Backend: Bestätigung erhaltener Config
smartboxes/{mac}/command  # Backend → SmartBox: Feuer-Kommando für ein Gerät

```

****Discovery-Payload**** (von SmartBox):

```
{ "mac": "aabbccddeeff", "firmwareVersion": "0.6", "boxType": "pico2w", "ip": "192.168.1.42" }
```

6. API-Features

Device Management (`/api/devices`, `/api/device-types`, `/api/smart-boxes`)

- CRUD für Geräte und Gerätetypen
- Pagination-Unterstützung
- Geräte-Kommandos, Range-Zuweisung
- Manuelle Config-Push zu SmartBoxen

Range Management (`/api/ranges`)

- Erstellen/Verwalten von Schiessbahnen
- Range-Sperre während aktiver Sessions
- Gerätezuweisung zu Ranges

Competition Sessions (`/api/sessions`) — Unique Feature

- Support für LEAGUE, BRACKET, KNOCKOUT Formate
- Gruppen-Management
- Bracket-Initialisierung mit mehreren Seeding-Strategien
- Leaderboard mit Ranking und Genauigkeits-Tracking
- Player Results Export

Real-Time Updates (`/api/events`)

- Server-Sent Events (SSE) Stream
- Events: SmartBox-Status, Config-Sync, Device-Health
- Clients abonnieren via JavaScript `EventSource` API

Authentifizierung (`/api/auth`)

- JWT-basiert (RS256)
- Benutzer-Rollen: ADMIN, SHOOTER
- Benutzer-Management Endpoints

7. Entwicklungs-Workflow

Alle drei Sub-Projekte folgen dieser Struktur:

Backend (`smart-ground-backend/`)

- **Stack**: Java 25, Spring Boot 4, PostgreSQL, Liquibase, Spring Integration
- **Package-Struktur**: api/ (REST), config/ (MQTT), dto/, model/ (JPA), service/
- **Konventionen**: UUID als PK, `@NullMarked` auf allen neuen Klassen, Deutsche Inline-Kommentare
- **Liquibase**: Disabled pre-v1.0 (JPA manages schema). At v1.0: enable und migrate.
- **Tests**: JUnit 5, H2 in-memory, embedded Mosquitto für Integration Tests

Frontend (`smart-ground-ui`)

- **Stack**: Vue 3 Composition API, Vite 8, Pinia, Node 20+
- **Konventionen**: Komponenten in `src/components/`, Stores in `src/stores/`, API-Calls über Service-Layer
- **Tests**: Vitest mit `@vue/test-utils`

SmartBox Firmware (`smart-box`)

- **Stack**: MicroPython 1.23+, Raspberry Pi Pico 2W, MQTT (umqtt.simple)
- **Memory**: Strict constraints (520 KB SRAM). `gc.collect()` nach jedem MQTT-Message.
- **Konventionen**: Deutsche Kommentare, English Identifiers, Safe-Exit Pattern in main.py

8. Status und Offene Punkte

■ Implementiert

- Geräte-Verwaltung (CRUD, Discovery, Config-Push)
- MQTT-basierte Kommunikation mit SmartBoxes
- REST-API mit OpenAPI-Spezifikation
- JWT-Authentifizierung und Rollen
- Wettkampf-Verwaltung (Session, Group, Leaderboard)
- SSE für Real-Time Updates
- Liquibase Migration-Strategie dokumentiert

■ In Progress / Zu klären

- **Authentifizierung / User Management**: Spring Security vorhanden, aber nicht konfiguriert
- **OTA Firmware-Updates**: Separate von der Fähigkeits-Registry
- **INPUT-Signal-Handling**: End-to-End (Mikrofon → Backend → Device auslösen)
- **Multi-SmartBox Device Assignment**: Datenmodell unterstützt es, API noch nicht exposed

■ Zukünftige Features (Post v1.0)

- **Multi-Anlagen / Mandantenfähigkeit**: Aktuell eine Anlage pro Installation
- **Hardware-Failsafe**: Physische Not-Aus-Taste (unabhängig von Software)

- **Offline-Robustheit**: Aktuell kein Service Worker, kein Offline-Mode (bewusst entschieden)
- **Rate Limiting**: Freie Nutzung – Max. 1 Wurf / 2 Sek pro Range?

9. Referenz-Dokumente

	Ort	Inhalt
	Root	Vollständige Projekt-Dokumentation mit Superpower
	ProjektManagement/	REST-Endpunkte + MQTT-Topic-Schema
	ProjektManagement/	Vollständiges PostgreSQL-Schema + Design-Hinw
	ProjektManagement/	Tech-Constraints, Naming Conventions, DE/EN-Ma
	ProjektManagement/	Domänen-Glossar, Rollen, Berechtigungen
	ProjektManagement/	Fachliche Abläufe und technische Kernabläufe

10. Quick-Start: Vollständiger Stack Lokal

```
# Terminal 1: Backend
cd smart-ground-backend
docker compose up # PostgreSQL + Mosquitto
./mvnw spring-boot:run -Dspring-boot.run.profiles=postgres

# Terminal 2: Frontend
cd smart-ground-ui
npm install
npm run dev # http://localhost:5173

# Terminal 3: SmartBox (optional, requires Pico board)
cd smart-box
# siehe smart-box/CLAUDE.md
```

Dokument aktualisiert: Mai 2026

Konsistent mit root CLAUDE.md (v1.0-dev)